分布式追加写存储系统

1. 背景
2. 架构设计
3. ChunkServer设计
  3.1. Chunk的读写流程
    3.1.1. 写流程
    3.1.2. 读流程
  3.2. 本地存储引擎
  3.3. Chunk丢失分片的修复
  3.4. 主动上报心跳
4. Master设计
  4.1. 元数据一致性协议
  4.2. 元数据存储
  4.3. 数据放置策略
  4.4. Chunk分片保活
5. Client设计
  5.1. ChunkClient设计
    5.1.1. 写逻辑
    5.1.2. 读逻辑
    5.1.3. 封存逻辑
    5.1.4. 黑名单
  5.2. FileClient设计

1. 背景

覆盖写存储系统

- 优势：空间利用率高(原地覆盖，不需要GC)，读取路径简单
- 劣势：数据一致性协议实现复杂，存储成本更高(对EC和LRC编码支持有限)

追加写存储系统：

- 优势：数据一致性协议实现简单，多版本和快照实现简单，存储成本更低
- 劣势：集群网卡带宽放大(GC)，磁盘带宽放大(HDD)，读取路径复杂(需要大量内存维护用户逻辑空间到存储物理空间的映射)。

由于现在网卡带宽越来越大、大内存的普及、以及SSD硬盘覆盖写也会导致盘上GC，考虑到追加写集群一致性协议实现简单，目前存储系统越来越多的选型追加写的方式。下面根据pangu论文的描述以及项目经验，总结该类型存储系统的一些共性。

2. 架构设计

ChunkServer：

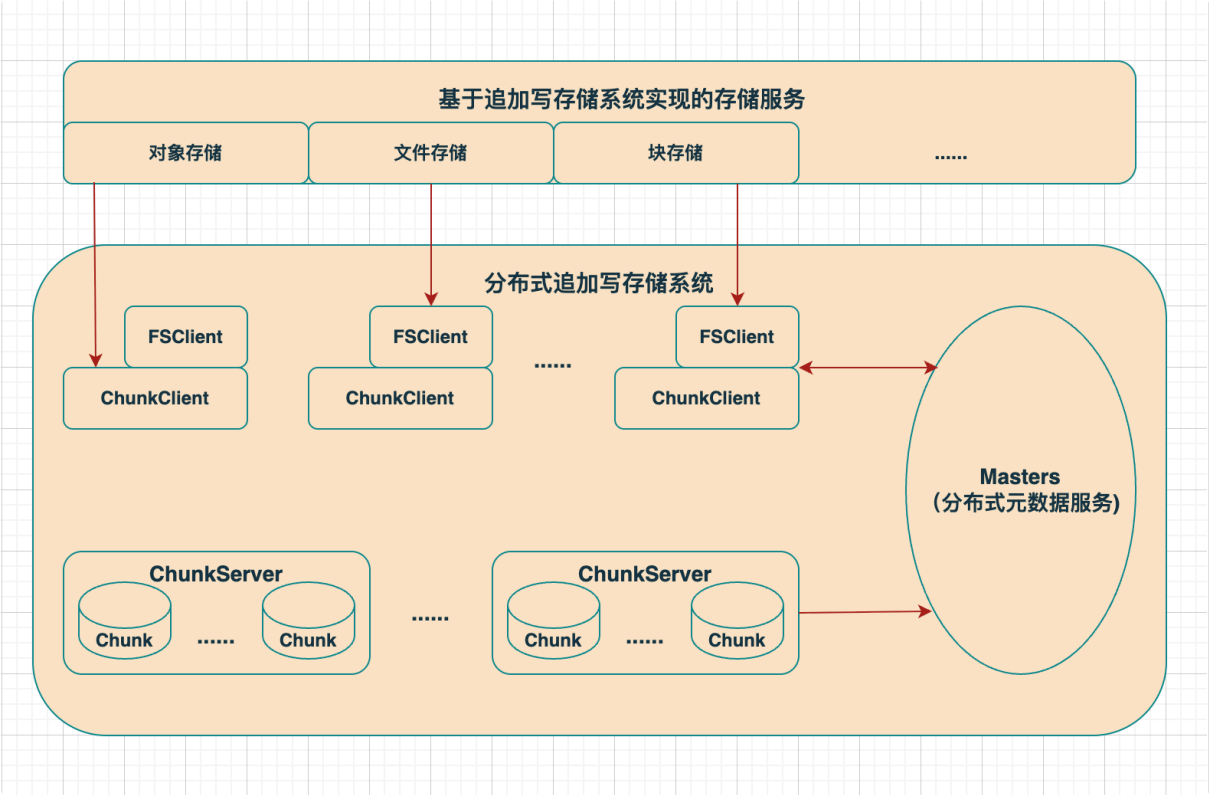
- 维护每个具体的Chunk分片的写入和读取。
- 执行修复

Master：

- 维护每个逻辑Chunk的相关信息，包括Chunk是否Seal，长度，采用的编码，每个分片的位置。
- 发起修复

Client：

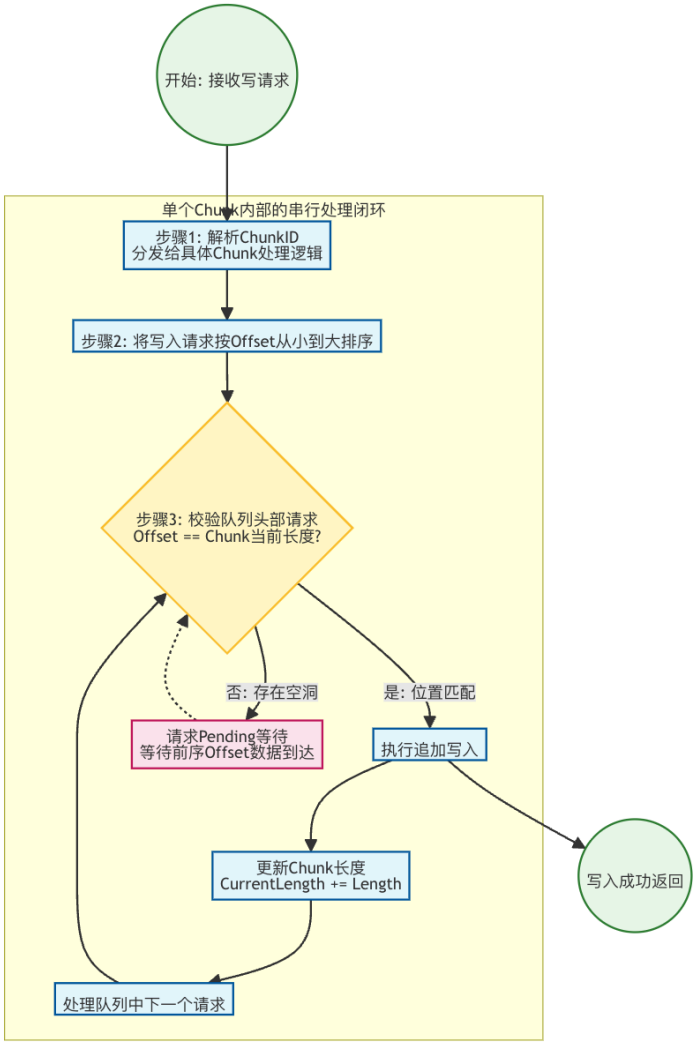
- 提供ChunkClient读写接口，主要给对象存储使用
- 提供FSClient读写接口，主要给文件存储，块存储使用。



3. ChunkServer设计

3.1. Chunk的读写流程

3.1.1. 写流程



- 写流程的核心是每个Chunk的写入必须是串行的，也就是该Chunk前面一笔数据写入成功后，才可以写入下一笔数据。
- 串行和同步之间没有耦合关系，异步串行写入和同步串行写入都可以，实际系统里面往往采用异步串行写入。
- 每一笔写入数据携带<Offset, Length>信息，写入之前需要校验一下数据offset位置是否是当前Chunk的长度，如果之间存在空洞，则写请求需要pending，等前面的请求到来。

3.1.2. 读流程

读流程根据读请求携带的<ChunkID, Offset, Length>，直接去对应位置读取数据即可

3.2. 本地存储引擎

本地存储引擎分为三种：

1. 基于本地文件系统的存储引擎
2. 基于裸盘的存储引擎(HDD)
3. 基于裸盘的存储引擎(SPDK+SSD)

方法1主要依赖本地文件系统对Chunk分片进行管理，方法2和3如果设计得当，在设备层之上逻辑是兼容的。

3.3. Chunk丢失分片的修复

Master把修复某个分片的任务发给Target节点，Target节点复用ChunkClient的读取逻辑，通过Decode逻辑恢复丢失数据，写入节点硬盘，从而实现对丢失数据的修复。

3.4. 主动上报心跳

ChunkServer需要主动周期性上报心跳给Master，Master依赖该心跳判断ChunkServer的健康状态。

4. Master设计

4.1. 元数据一致性协议

Master本质是一个分布式元数据服务，一般采用raft协议。

- 一组raft：元数据规模受单台机器的处理能力限制，但是实现OP之间的原子性非常简单
- 多组raft：元数据规模不受单台机器的处理能力限制，但是跨raft组实现OP之间的原子性比较难，需要分布式事务。

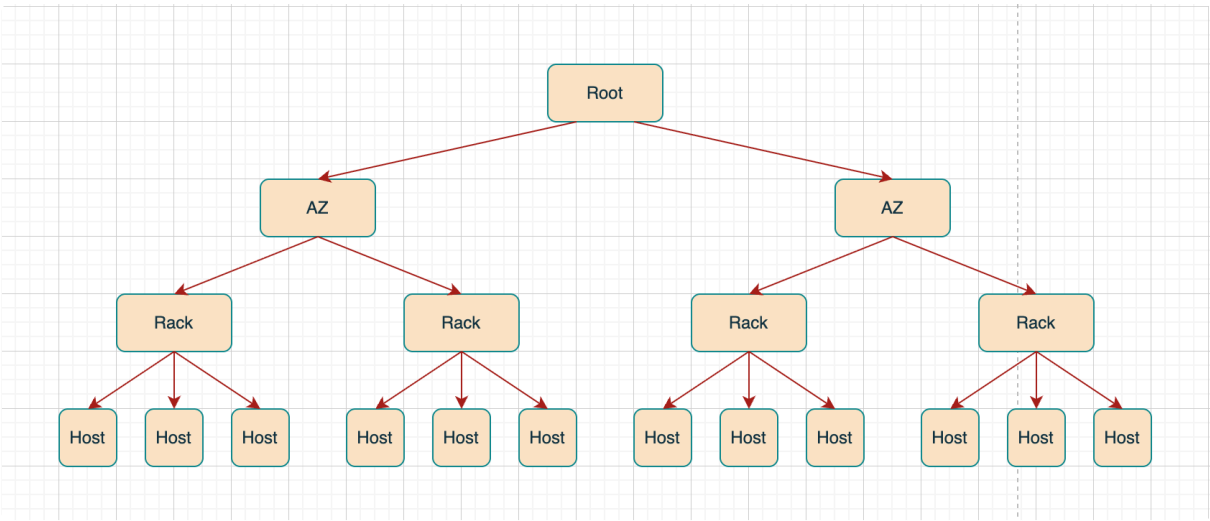
目前主流的设计采用一组raft实现，限制单个Chunk大于一定容量(512MB)，从而实现单个存储集群规模到PB级别。

4.2. 元数据存储

元数据类型	描述
ChunkInfo	Chunk的基本信息(状态，长度)和分片路由信息
ChunkServerInfo	ChunkServer节点基本信息(状态，IP, Port)
ConfigInfo	集群的配置信息

4.3. 数据放置策略

数据可以根据选择的故障阈(Host， Rack， AZ级别)，把某个Chunk的多个分片放置在不同的Host或者Rack或者AZ中。



4.4. Chunk分片保活

Master 主节点会周期性去ChunkServer节点收集Chunk分片信息，超过几轮未收集到某分片，即认为该分片失联，从而发起对该分片的修复任务。

5. Client设计

该架构属于重客户端设计，负责处理Chunk多分片的编解码，读写一致性逻辑。

5.1. ChunkClient设计

- Chunk的写入采取强一致性写入策略，即Chunk所有分片都写入成功才算成功。
- 当某个Chunk分片写入失败，该Chunk就无法继续写入，客户端创建一个新Chunk替代，同时把上一个写入失败的Chunk进行Seal

5.1.1. 写逻辑

一笔数据追加写入Chunk，根据编码格式把数据分成各个分片，然后把所有分片数据下发ChunkServer写入，等所有ChunkServer写入成功返回后，则认为该笔数据成功写入。

5.1.2. 读逻辑

根据读取数据的<offset, length>，计算出数据分散在哪些分片上，对数据所在的分片发起读取，如果超过RTT时间后数据没有全部返回，则说明数据分片有可能丢失，然后发起BackUp读，读取所有校验分片，通过decode计算出丢失数据，然后返回给用户成功。

5.1.3. 封存逻辑

当Chunk写入失败后，Seal核心是告诉Master该Chunk成功写入的长度，以及封存Chunk的分片，阻止后续的写入。

5.1.4. 黑名单

根据读写失败请求的Target IP，生成一个节点黑名单。

- 读请求如果发现某个数据分片在黑名单中，则可以直接发起校验分片的读取
- Chunk写入失败后，重新创建Chunk的时候，可以把写入失败的IP携带给Master，让Master避免在这些IP上面放置新Chunk的分片。

5.2. FileClient设计